CLASSIC SNAKE â GAME DESIGN BRIEF

Author: Codex (GPT-5) ´• Repo: snake-game ´• Date: February 2, 2026

1. Vision & Pillars

Deliver a timeless Snake experience that feels immediate, readable, and responsive inside a single-page web app. The build must stay dependency-light, deterministic, and keyboard-first while providing mobile-friendly controls.

Pillars

- â¢ Deterministic logic â pure state transitions in SnakeGame enable straightforward tests and debugging.
- â¢ Minimalist UI â single grid, HUD, overlay, and controls live in one stylesheet.
- â¢ Inclusive input â arrow keys, WASD, tap buttons, and auto-pause for focus changes.

Metric | Target | Rationale

Perceived responsiveness â ~60 FPS feel via 140 ms ticks â Keeps motion lively without overwhelming casual players.

Dependencies â Zero runtime packages â Matches repo scope and enables static hosting anywhere.

2. Core Loop

Four deterministic phases run each tick:

1. Input â queue direction changes unless reversing into the snake.
2. Advance â promote next direction, move head, shift body.
3. Collide â detect walls/self; end run and freeze loop if triggered.
4. Resolve â grow and score when food matches head, otherwise trim tail.

Every tick returns a snapshot consumed directly by the renderer, so DOM updates always reflect authoritative state.

3. Board, Speed & Progression

Setting | Value | Rationale

Grid Size â 20 ˆ 20 cells â Balances open space with quick difficulty ramp as the snake lengthens.

Tick Interval â 140 ms â Fast enough to feel lively while remaining readable on touch devices.
Initial Body â 3 segments â Provides maneuverability yet shows growth immediately.
Food RNG â Clamped Math.random() â Guarantees spawns stay on-grid and avoid snake tiles.

These knobs live inside SnakeGame, making difficulty tuning centralized and testable.

4. Systems Architecture

4.1 State Engine (src/snakeGame.js)

- â¢ Stores snake array, food position, direction queue, score, status, and best score.
- â¢ tick() mutates internal arrays but exposes a cloned snapshot to presenters.
- â¢ Collision helpers keep logic tiny and deterministic; RNG injection enables repeatable tests.

4.2 Presenter (src/main.js)

- â¢ Bootstraps DOM grid once, keeping references in a Map for O(1) lookups.
- â¢ Keyboard handler maps Arrow keys and WASD; tap buttons emit the same direction events.
- â¢ Pause/Resume toggle the interval without destroying game state; overlay communicates end-state.
- â¢ Window blur auto-pauses; focus resumes to protect players from background losses.

4.3 Persistence

Session-only best score is stored in memory; extendable to localStorage if requirements expand.

5. Controls & UX

- â¢ Keyboard: Arrow keys & WASD prevent reversal into the snake body within one tick.
- â¢ On-screen D-Pad: Touch-friendly buttons mirror keyboard events with touchstart preventing scroll.
- â¢ Auto Pause: Window blur triggers pause; focus resumes automatically.

â¢ HUD: Score, Best, Status, Pause, and Restart; overlay reinforces end state with a restart CTA.

6. Visual Language

Styling relies on a single styles.css file:

â¢ Soft blues and warm oranges differentiate snake vs. food with accessible contrast.

â¢ Grid squares use the padding-top trick for perfect ratios and subtle gradients for depth.

â¢ Overlay uses translucent navy with centered text to spotlight the restart action.

7. Quality Plan

Automated Tests (tests/logic.test.js)

â¢ Movement & default direction progression.

â¢ Reverse-direction guard to avoid instant self-collisions.

â¢ Growth & scoring when food aligns with next head cell.

â¢ Wall-collision detection and status updates.

â¢ Deterministic food placement via stubbed RNG.

Manual Checklist

â¢ Play with keyboard and tap controls; confirm latency-free steering.

â¢ Toggle Pause/Resume while in motion and after blur/focus events.

â¢ Trigger wall and self collisions; overlay + best-score update must appear.

â¢ Verify food never overlaps the snake or spawns outside the grid.